LABORATORIO DE SERVICIOS DE ANALÍTICA EN AWS

Este laboratorio práctico de servicios de analítica de AWS para principiantes, busca comenzar con ejercicios separados por servicio, ya que nos permite construir una sólida comprensión de cada servicio de forma independiente.

VENTAJAS:

Aprendizaje enfocado: Los estudiantes pueden sumergirse profundamente en cada servicio sin distracciones.

Solución de problemas simplificados: Si surgen problemas, son más fáciles de aislar y abordar.

Aprendizaje progresivo: Permite a los estudiantes construir un conocimiento fundamental de cada servicio antes de combinarlos.

EJERCICIOS A REALIZAR:

AWS GLUE: Extraer, transformar y cargar datos.

AMAZON ATHENA: Consultar datos almacenados en S3.

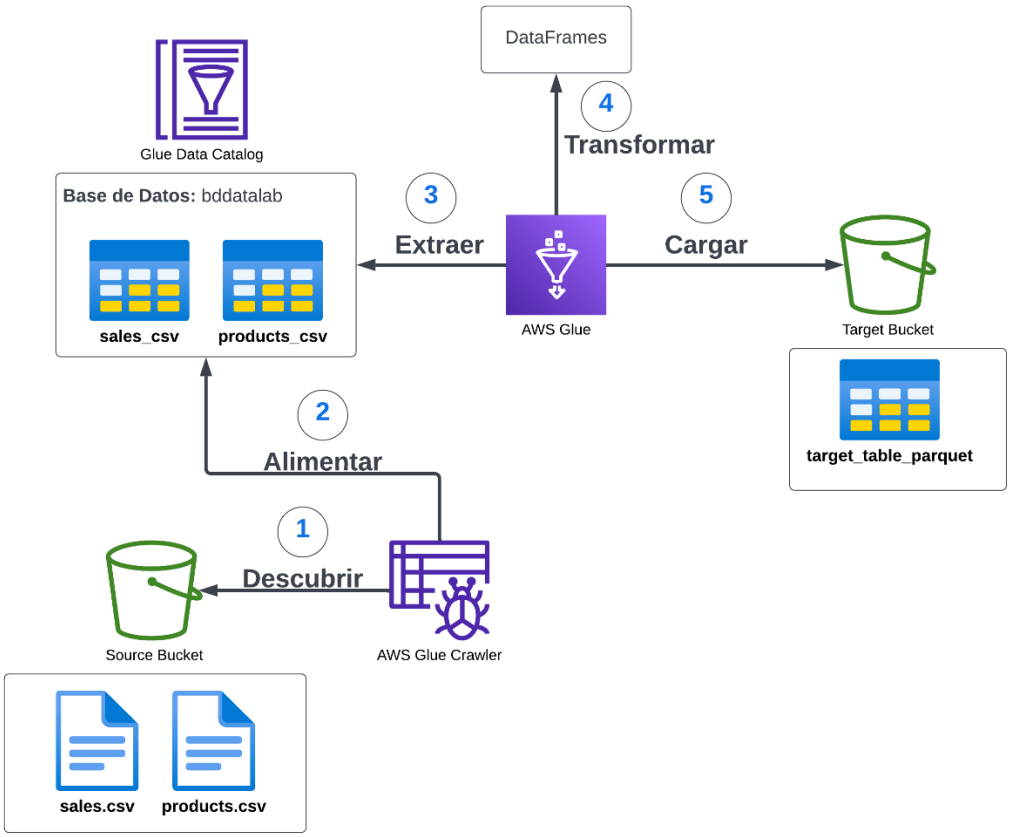
AMAZON QUICKSIGHT: Crear visualizaciones y paneles de datos.

Laboratorio de AWS Glue y Amazon Athena

Este código lee desde el AWS Glue Datacatalog dos tablas en una base de datos, después realiza un join (unión) entre ambas tablas y calcula el total de la venta. Finalmente, escribe el resultado en un archivo Parquet en S3.

En este laboratorio vamos a probar en la práctica qué es una ETL, es decir, qué es un proceso de extracción, transformación y carga, en el cual vamos a cargar unos datos a un bucket de S3, los vamos a transformar, luego los cargamos en otro bucket S3 y finalmente, los consultamos con Amazon Athena.

Arquitectura:



Servicios a utilizar:

Amazon Athena.
AWS Glue.
AWS Glue Data Catalog.
AWS Glue Crawler.
Amazon S3.
IAM.

Conceptos clave:

- **Dynamic Frame:** Es una estructura de datos similar a un DataFrame de Apache Spark, pero con características adicionales que lo hacen más adecuado para los procesos de transformación de datos en AWS Glue.

- **Data Frame:** Un DataFrame es una estructura de datos etiquetada bidimensional con columnas de tipos potencialmente diferentes. Puede pensar en un DataFrame como una hoja de cálculo, una tabla SQL o un diccionario de objetos de serie. DataFrame de Apache Spark proporciona un amplio conjunto de funciones

(selección de columnas, filtro, unión, incorporación) que permiten resolver problemas comunes de análisis de datos de forma eficaz.

- **Inner Join:** Es un tipo de operación de combinación (join) en bases de datos y sistemas de procesamiento de datos (como SQL, Spark, etc.) que se utiliza para combinar filas de dos tablas basándose en una condición común.

Explicación del código:

- Lee datos desde el AWS Glue Data Catalog, sales_csv y products_csv son leídos de la base de datos bddatalab. Estos sets de datos son cargados en Dynamic Frames:

```
dyf_sales = glueContext.create_dynamic_frame.from_catalog(
   database="bddatalab",
   table_name="sales_csv",
   transformation_ctx="dyf_sales"
)
dyf_products = glueContext.create_dynamic_frame.from_catalog(
   database="bddatalab",
   table_name="products_csv",
   transformation_ctx="dyf_products"
)
```

- Los Dynamic Frames se convierten en DataFrames para ser manipulados fácilmente:

```
df_sales = dyf_sales.toDF()
df_products = dyf_products.toDF()
```

- Se hacen las siguientes transformaciones: La columna "quantity" se convierte en un entero y la columna "price" es convertida en un flotante:

```
df_sales = df_sales.withColumn("quantity", col("quantity").cast("int"))
df_sales = df_sales.withColumn("price", col("price").cast("float"))
```

- Los DataFrames df_sales y df_products se unen según a través de la columna product_id mediante un inner join:

```
df_joined = df_sales.join(df_products, on="product_id", how="inner")
```

- Se agrega una nueva columna al DataFrame llamada "total_amount" multiplicando el valor de "quantity" y "price":

```
df_transformed = df_joined.withColumn("total_amount", col("quantity") * col("price"))
```

- El DataFrame es convertido nuevamente en un DynamicFrame para ser escrito en Amazon S3:

```
dyf_transformed = DynamicFrame.fromDF(df_transformed, glueContext,
"dyf_transformed")
```

- El DynamicFrame es escrito en Amazon S3 en el bucket bootcamp-lab-target en formato parquet:

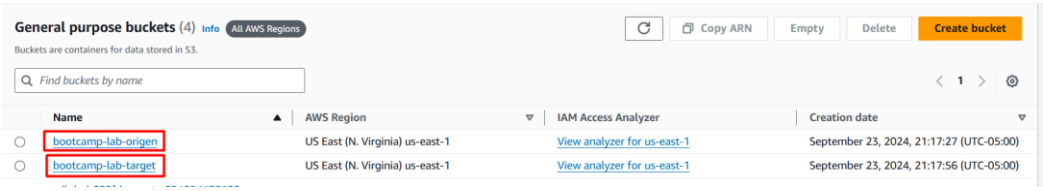
```
glueContext.write_dynamic_frame.from_options(
   frame = dyf_transformed,
   connection_type = "s3",
   connection_options = {"path": "s3:// bootcamp-lab-target"},
   format = "parquet",
   transformation_ctx = "write_parquet"
)
```

- Se escriben todos los cambios hechos de forma satisfactoria:

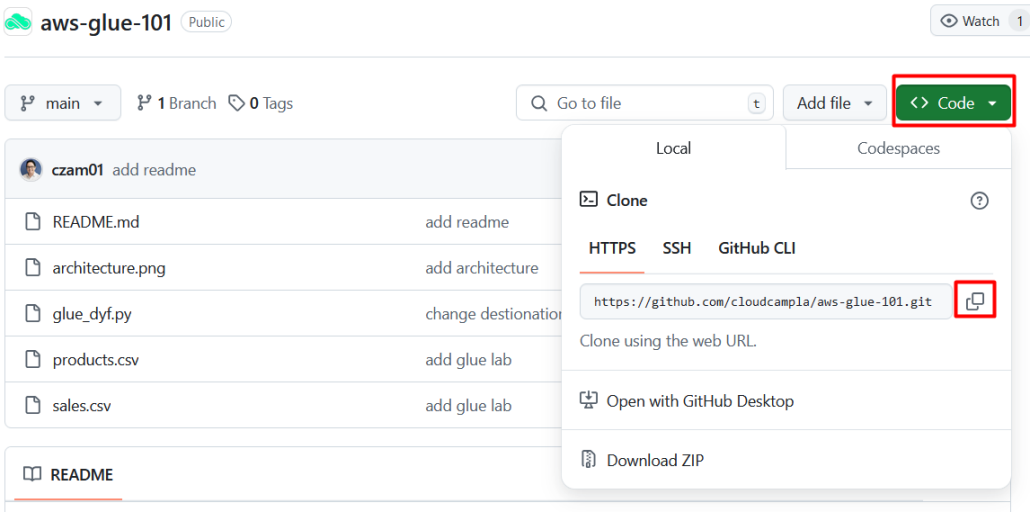
```
job.commit()
```

Pasos:

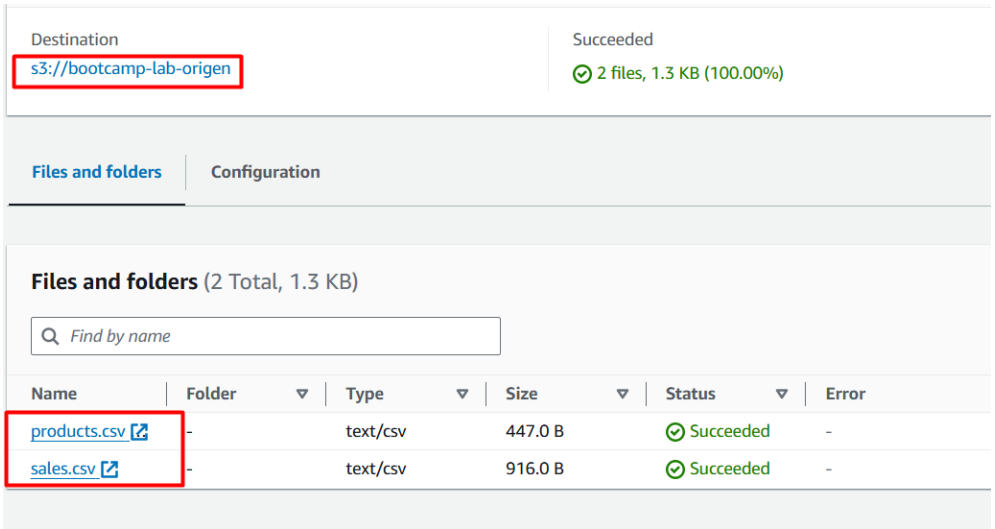
1. Crear en Amazon S3 el bucket origen y el bucket destino: En la consola de Amazon S3 damos clic en Create bucket, ponemos el nombre bootcamp-lab-origen, dejamos las configuraciones por defecto y al final damos clic en Create bucket.
   Repetimos el paso para crear otro bucket con el nombre bootcamp-lab-target:



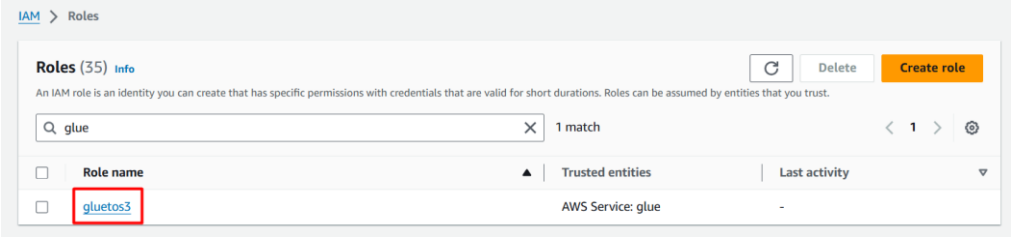
2. Clonar el repositorio https://github.com/cloudcampla/aws-glue-101.git:



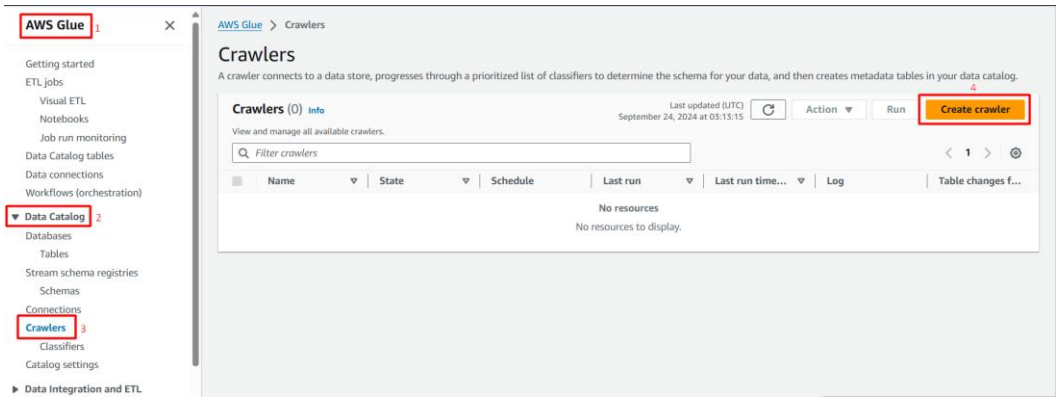
3. Cargar los archivos products y sales, descargados en el paso anterior, al bucket origen:



4. Crear rol IAM: Crear rol de servicio de Glue con permisos sobre S3:

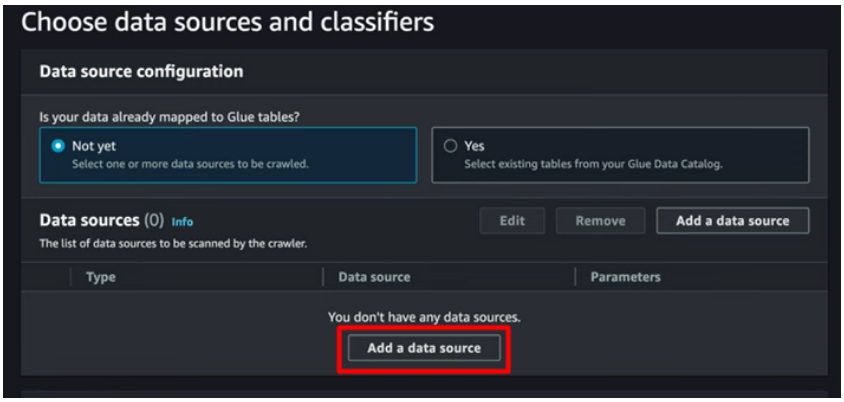


5. Crear crawler de Glue: Agregamos un rastreador que vaya e identifique esos dos archivos:

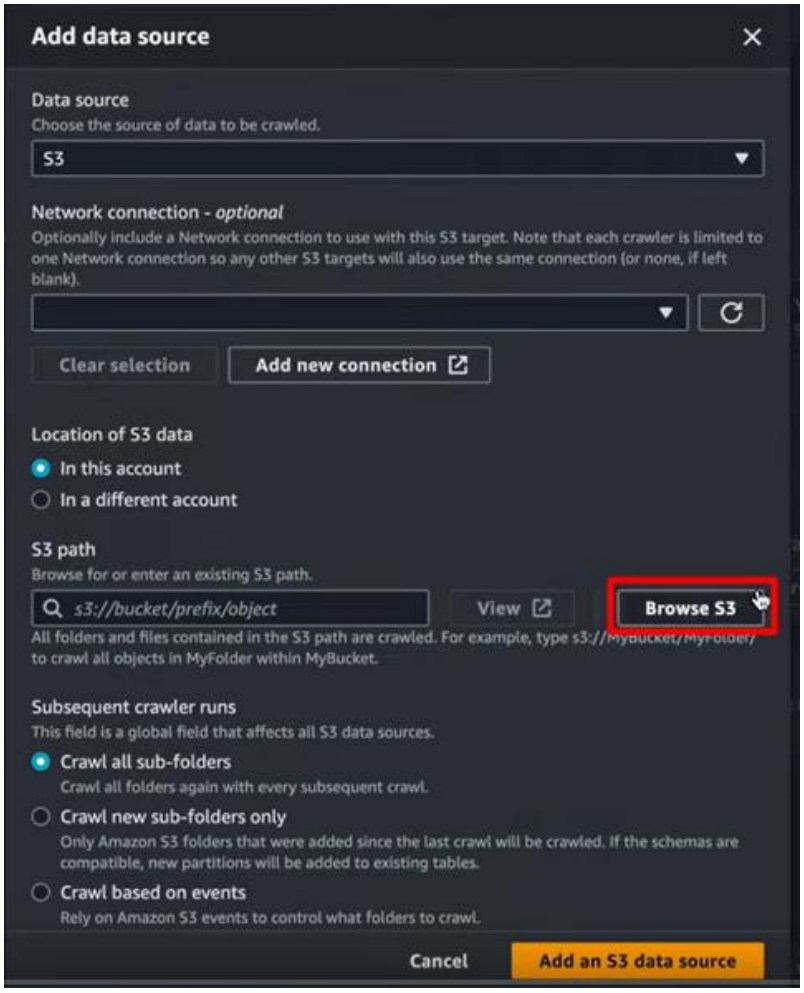


Asignamos un nombre y una descripción al crawler, en este caso ponemos
Nombre: crawlerbootcamp
Descripción: Identificar la estructura de fuente de datos del Bucket de origen.
Clic en Siguiente.

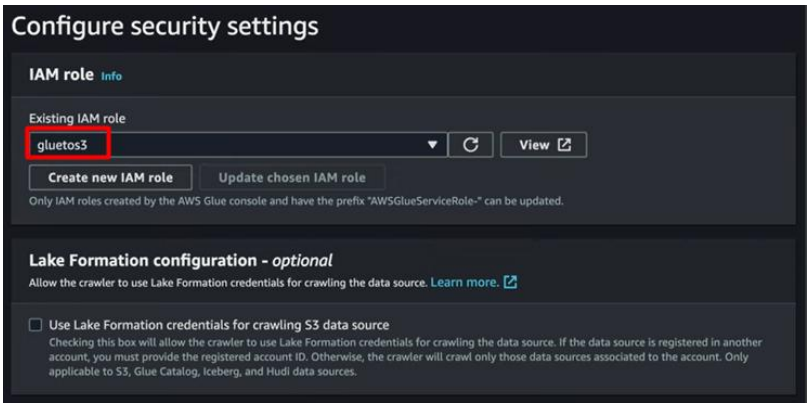
A continuación, debemos definir cómo lo vamos a mapear con el Data Catalog: Damos clic en Add a data source, es decir, a dónde va a ir el crawler para buscar esa data de origen



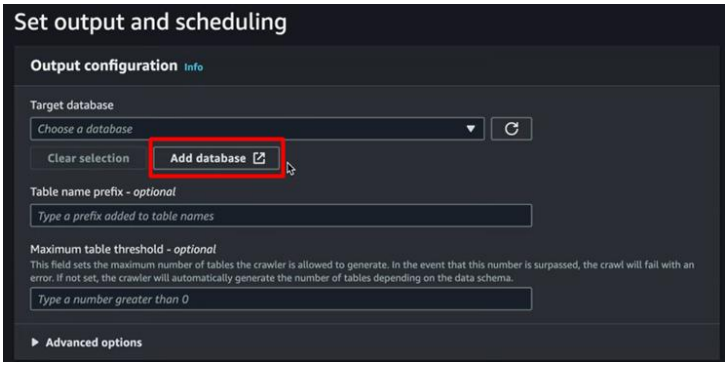
Nos pide un S3 path, damos clic en Browse S3 y seleccionamos nuestro bucket origen. Dejamos seleccionado Crawl all sub-folders para indicarle que haga un rastreo de todas las subcarpetas y damos clic en Add an S3 data source:



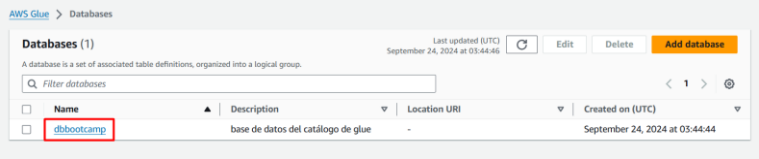
Seleccionar el rol que creamos en el punto 4 y damos clic en Siguiente:



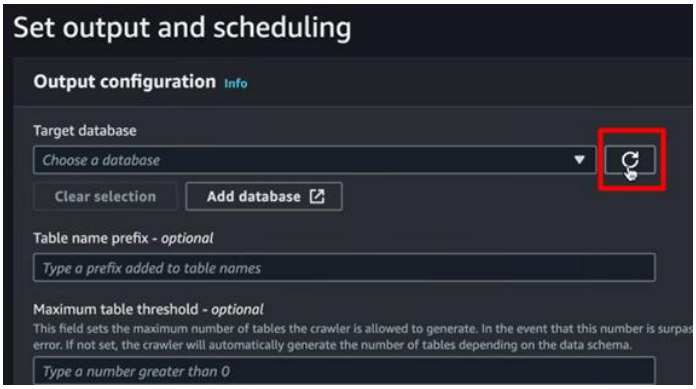
Crear base de datos en el catálogo de Glue: En qué base de datos del Glue Data Catalog se van a poner los datos rastreados, como aún no tenemos ninguna, procedemos a crearla dando clic en Add database, le asignamos el nombre *dbbootcamp*, en la descripción *base de datos del catálogo de glue* y clic en Create database:



Base de datos creada en el Data Catalog:

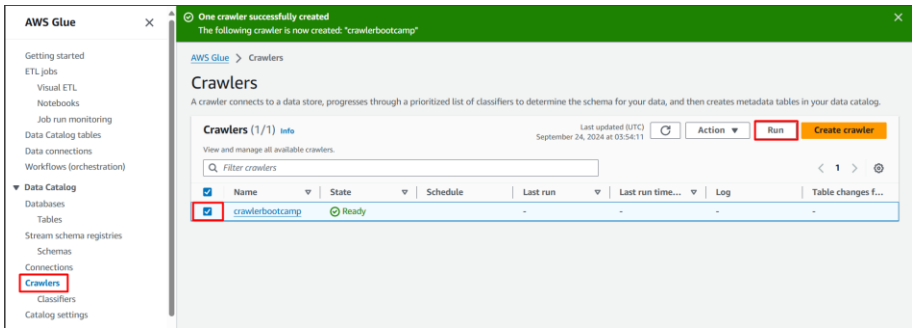


Volvemos a la consola de Glue, damos clic en actualizar y nos trae la BD que acabamos de crear en el Glue Data Catalog, la seleccionamos, no le ponemos prefijo a las tablas y en la programación del rastreador (Crawler Schedule) lo dejamos a que corra por demanda, podemos establecerle un horario, pero como para este laboratorio solo lo vamos a ejecutar una vez, lo dejamos On demand y damos clic en Next o Siguiente:



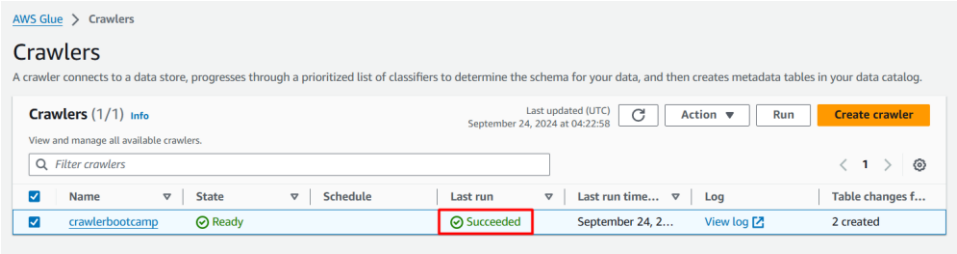
Finalmente, damos clic en Create Crawler.

6. Seleccionamos el crawler que acabamos de crear y le damos Run para que se ejecute:



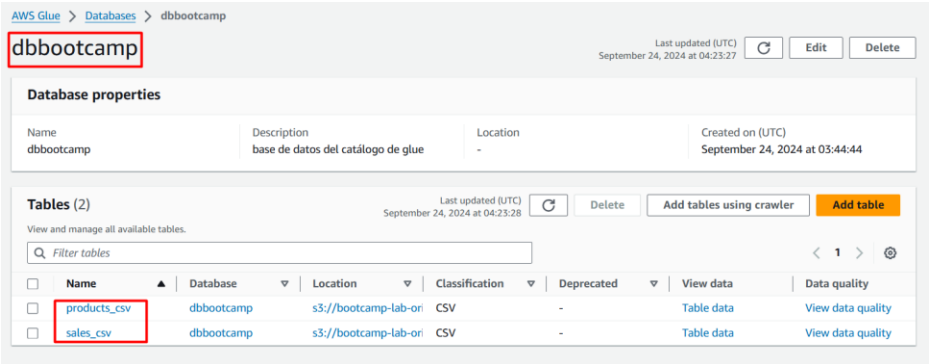
Este rastreador tiene un rol con permisos sobre S3, dicho rol entra al bucket que le especificamos, lee los csv, identifica la estructura de esos archivos y los carga como tablas en la base de datos que creamos en el catálogo de Glue.

Esperamos mientras finaliza la ejecución de forma exitosa:



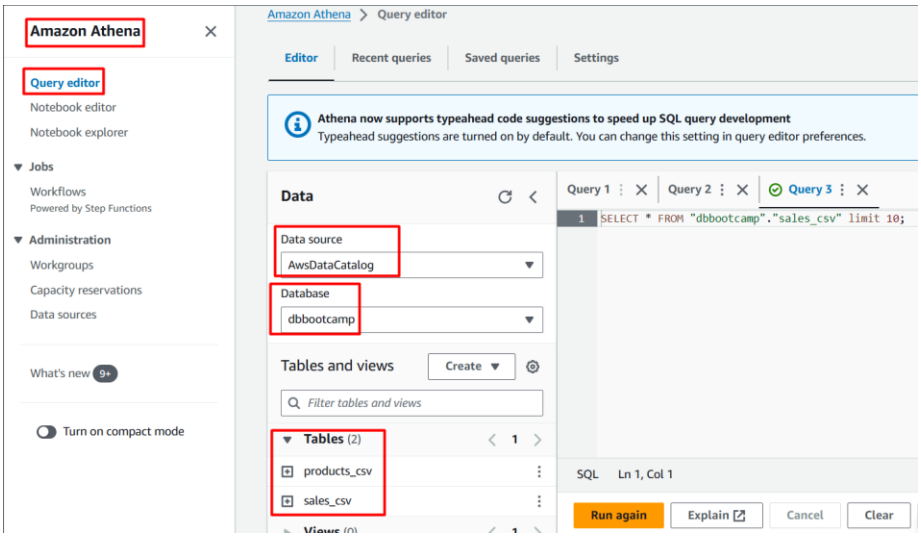
Luego de descubrir la data, el crawler alimenta la base de datos del datacatalog bdbootcamp, con las tablas que encontró de los dos archivos que tenemos en el bucket S3.

Una vez termina de ejecutarse, revisamos la base de datos y vemos que el crawler nos creó dos tablas:

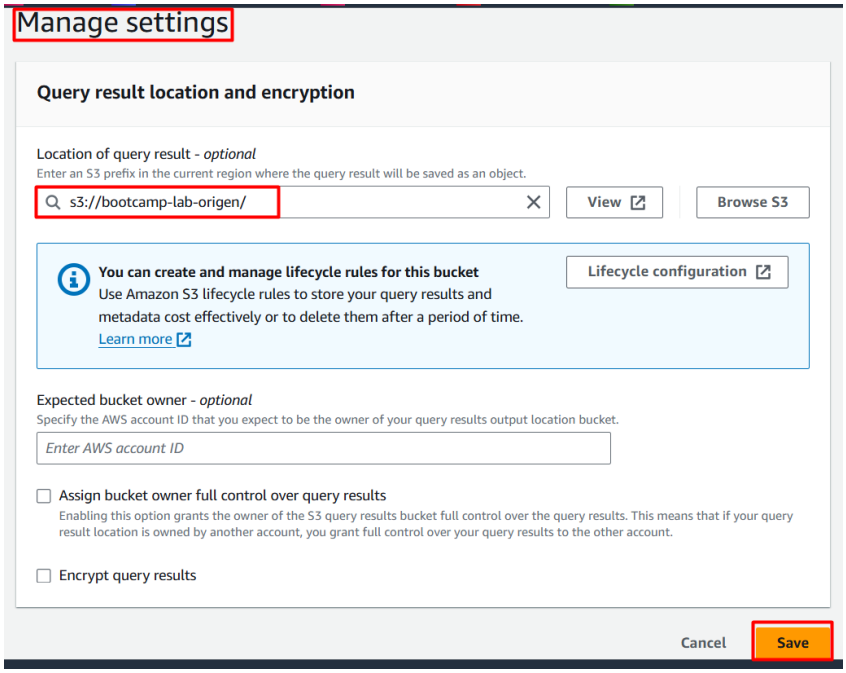


7. Consultar la data con Athena: La data cargada en el catálogo de Glue la podemos consultar con Amazon Athena, que es un motor de consultas que va a leer el catálogo de datos de Glue.

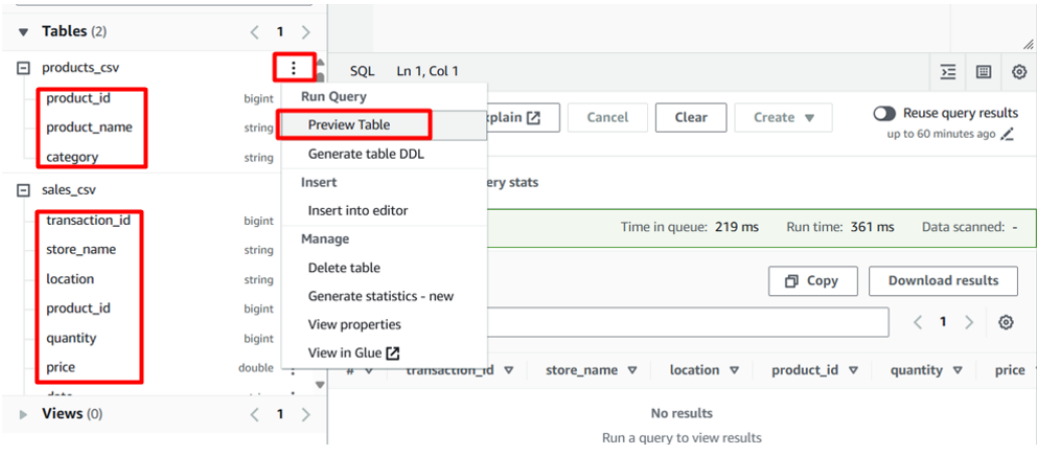
   En la consola de Amazon Athena, en el panel del lado derecho, seleccionamos Query editor y vemos el data source, la base de datos y las tablas:



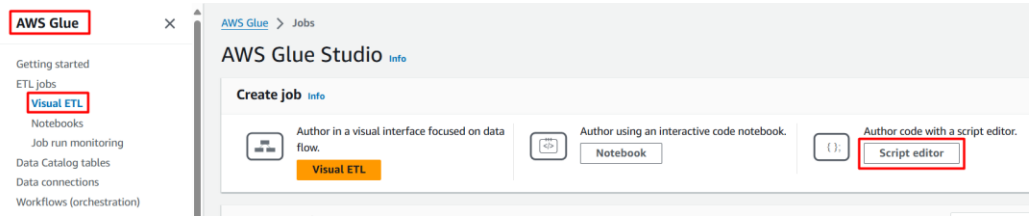
   Editamos las configuraciones para indicarle el bucket en el que va a quedar el resultado de las queries, damos clic en browse y podemos seleccionar el bucket origen:



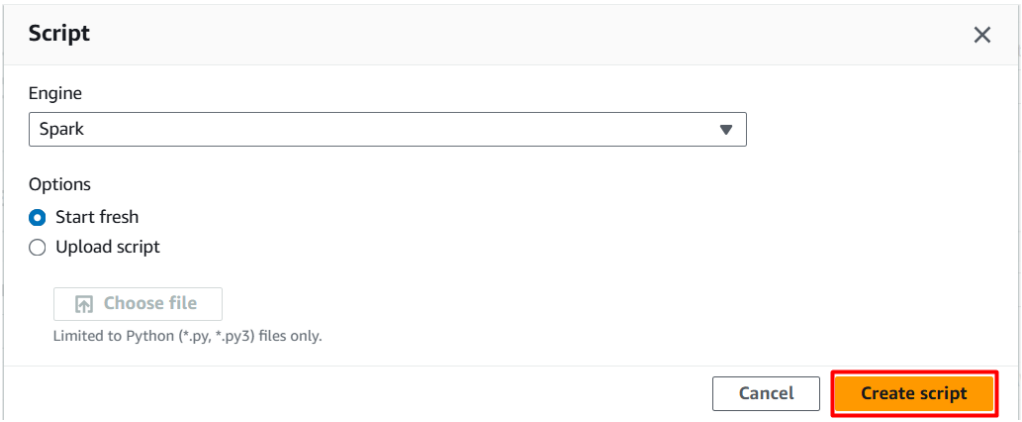
Regresamos al Query editor, damos clic en los 3 puntos de la tabla products, seleccionamos Preview Table y podemos identificar la data que tenemos guardada, ya nos carga el producto_id, el product_name y el category de la tabla products, de igual forma, nos carga la data de la tabla sales:



8. Cargar job y hacer la transformación: En la consola de AWS Glue, seleccionamos Visual ETL y damos clic en Script editor:



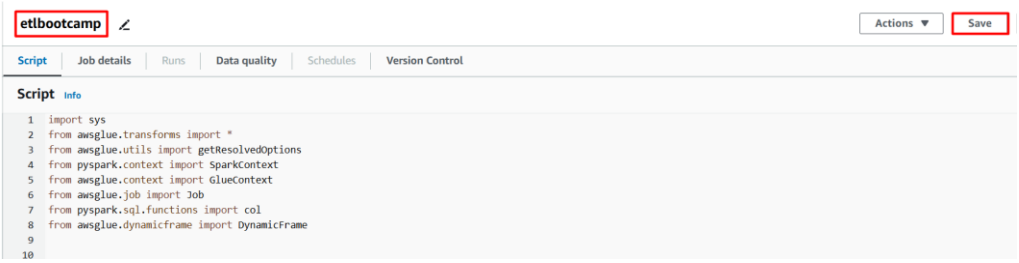
Aquí vamos a configurar el job con el código que descargamos del repositorio de GitHub, damos clic en Create script y se crea un job nuevo:



Aquí vamos a configurar el job con el código que descargamos del repositorio de GitHub, damos clic en Create script y se crea un job nuevo.
Abrimos el archivo glue_dyf.py, copiamos el contenido y reemplazamos el código, teniendo en cuenta modificar las siguientes líneas:

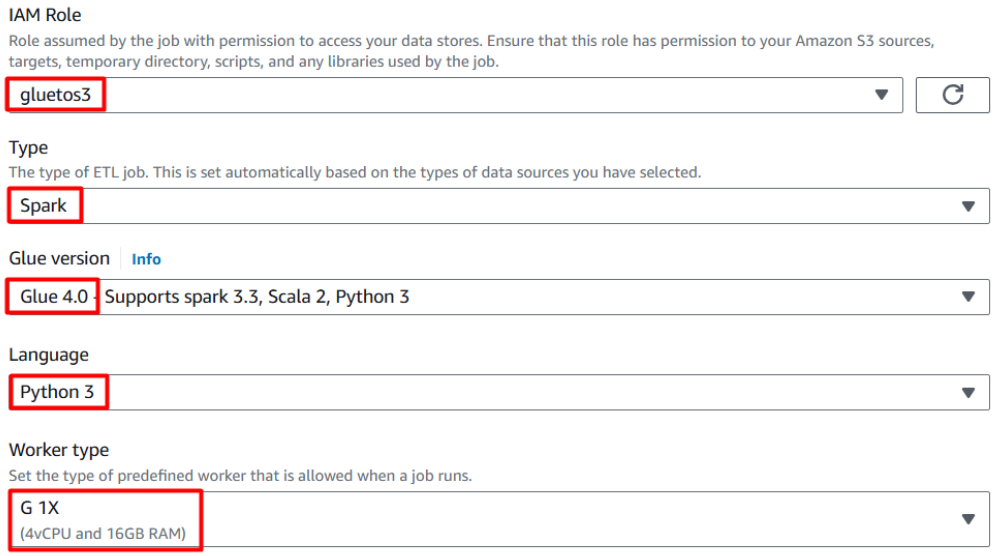
Línea 25 database = dbbootcamp
Línea 31 database = dbbootcamp
Línea 50 bucket destino = bootcamp-lab-target

Le asignamos un nombre, para este caso etlbootcamp y damos clic en Save:

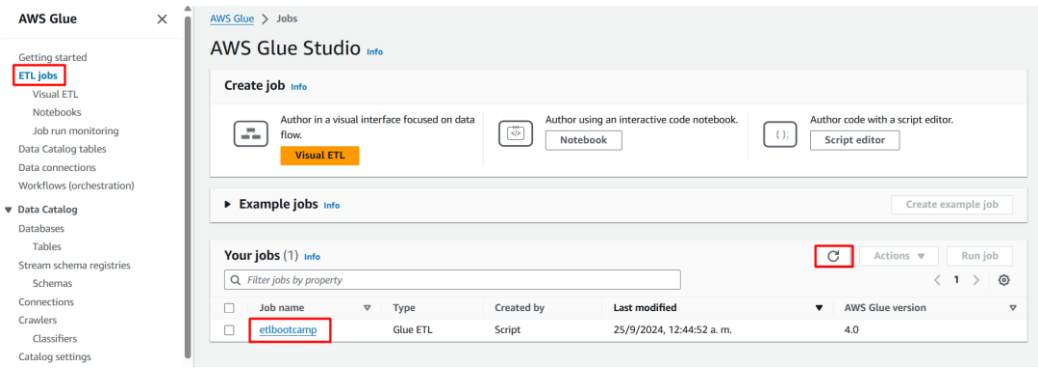


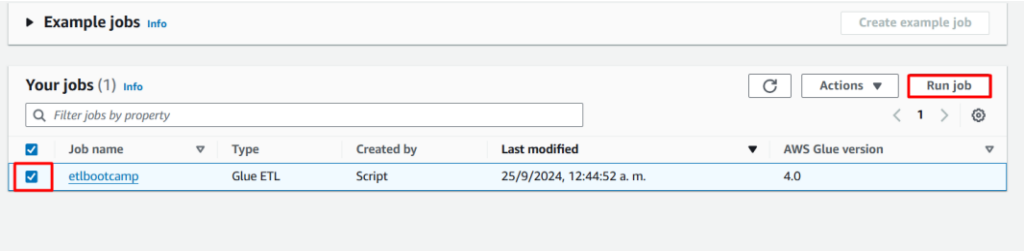
En Job details:

Descripción: ETL para hacer una transformación de csv y dejarla en S3 en formato parket.
Escogemos el rol que tiene permisos sobre S3 para leer y escribir la data.
Tipo de ETL ***Spark*** en su versión ***4.0***.
Lenguaje ***Python 3***.
Worker type: Tipo de worker que va a ejecutar esta ETL, dejamos ***G 1X*** que es el más pequeño porque para este caso no necesitamos más.
Requested number of workers: Cuántos workers necesitamos, como esto es una transformación sencilla de ejemplo, ponemos ***2*** que es el mínimo de workers.
Job bookmark: Se utiliza cuando queremos marcar hasta dónde se ejecutó el job y cuando vuelva a ejecutarse se vuelva a ejecutar desde esta marca, en este caso lo dejamos en ***Disable***.
Dejamos el resto de la configuración por defecto y damos clic en la parte superior en Save:

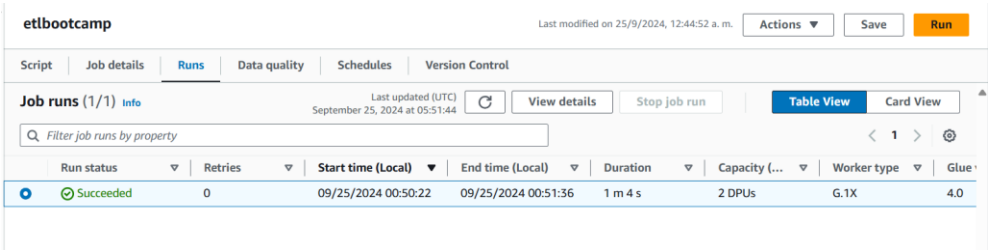


Regresamos a ETL Jobs, actualizamos y ya nos carga el job que acabamos de crear, lo seleccionamos y lo ejecutamos:

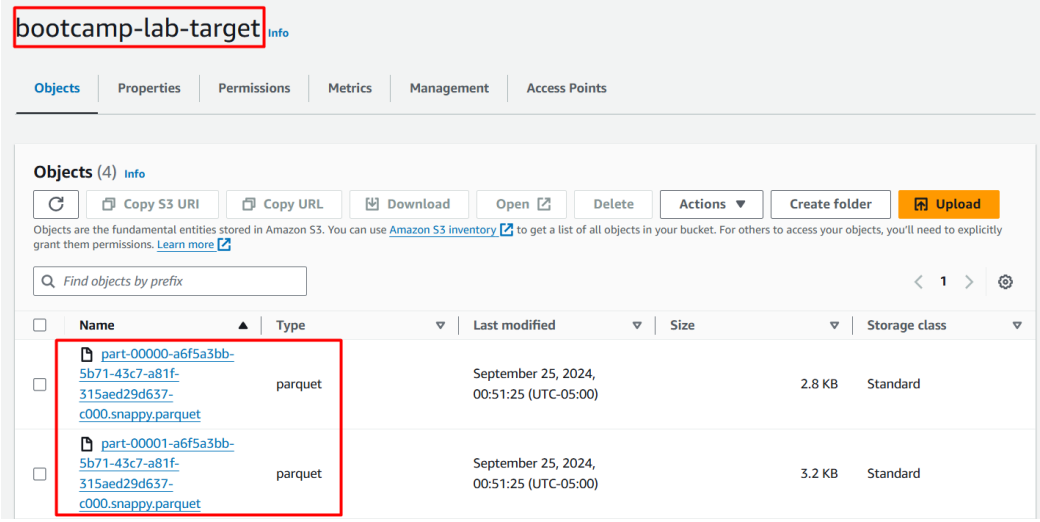




Esto toma unos minutos mientras inicializa el worker, carga el código y comienza la transformación.
Cuando se ejecuta ingresamos a él, damos clic en Runs y nos muestra la información del job:

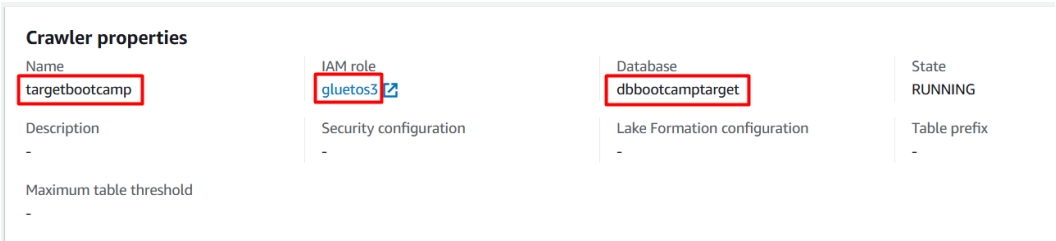


En Amazon S3 ingresamos al bucket bootcamp-lab-target y ya tenemos los archivos en formato parquet:

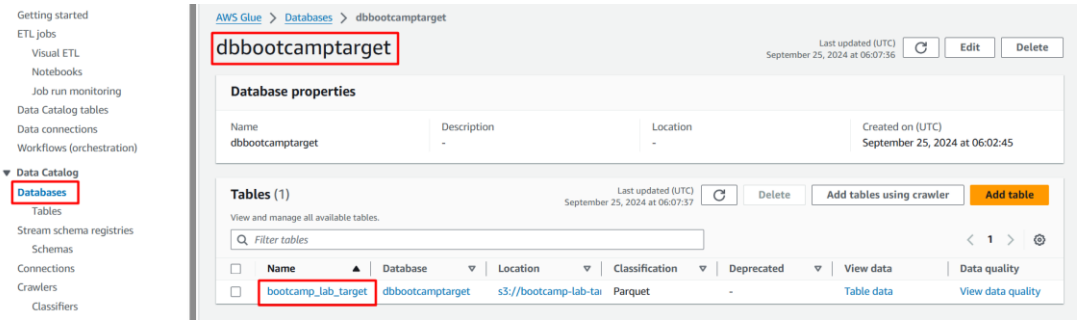


Los pasamos a formato Parquet porque éste es un formato de archivo más eficiente en cuanto a compresión y para que otros servicios puedan usarlo de forma más rápida, en comparación con un archivo en csv.

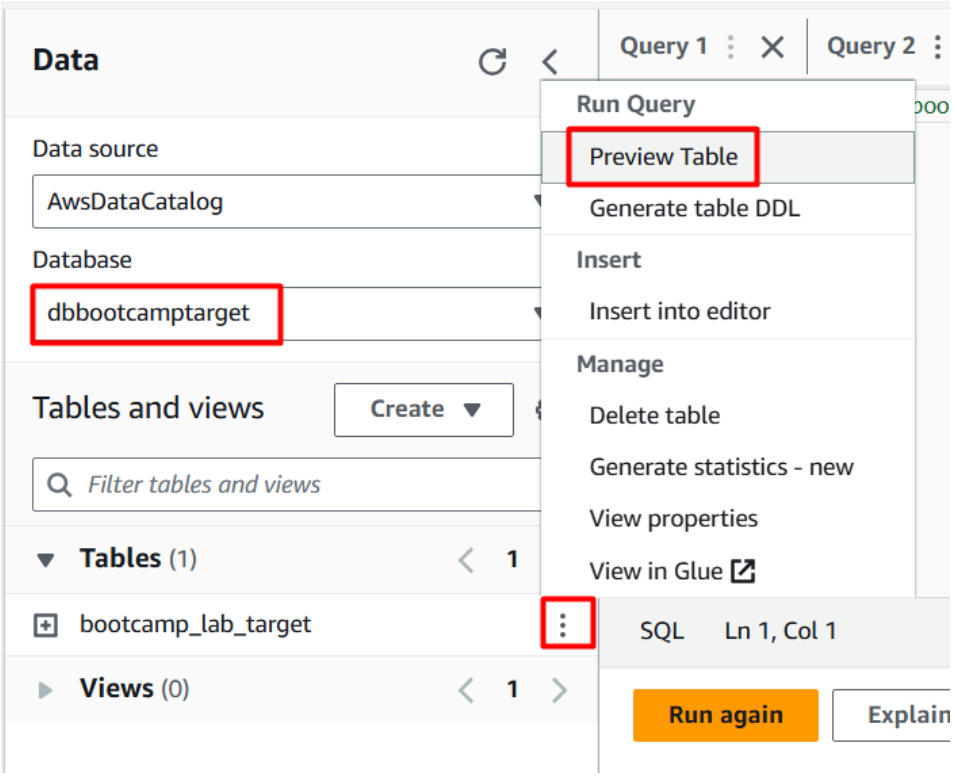
9. En AWS Glue vamos a crear otro crawler para que detecte esos archivos en parquet del destino: Le asignamos el nombre targetbootcamp, seleccionamos el mismo rol, creamos la base de datos destino dbbootcamptarget, dejamos el resto de configuraciones por defecto y damos clic en Create crawler:



Luego de crear el crawler lo ejecutamos dando clic en Run crawler, cuando termina la ejecución del crawler vamos a Data Catalog y ya vemos la base de datos creada con una tabla destino:



10. En Amazon Athena, seleccionamos la base de datos dbbootcamptarget, damos clic en los tres puntos de la tabla bootcamp_lab_target y seleccionamos Preview Table:



Aquí podemos ver que, de acuerdo a las especificaciones del job, se hizo la transformación de los campos y el join de las tablas como se le pidió en el script:

Query results | Query stats

Completed — Time in queue: 67 ms — Run time: 465 ms — Data scanned: 2.02 KB

Results (10) — Copy — Download results

Search rows — 1

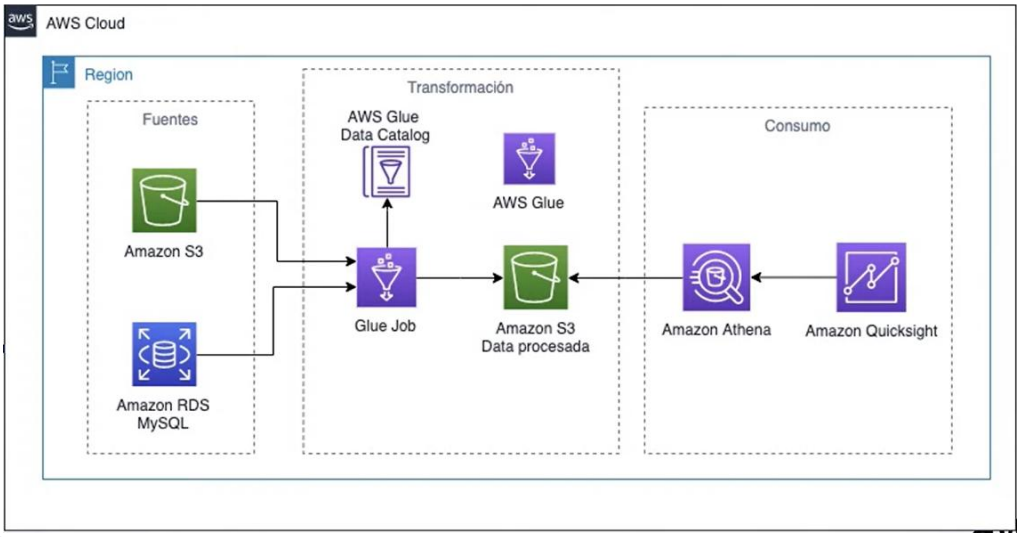
#	product_id	transaction_id	store_name	location	quantity	price	date	product_na
1	101	1001	Tech Haven	New York	15	199.99	2023-01-15	4K Ultra HD
2	101	1003	Electro Central	Los Angeles	10	199.99	2023-01-17	4K Ultra HD
3	102	1002	Tech Haven	New York	7	299.99	2023-01-16	Smartphon
4	102	1006	Digital World	Chicago	4	299.99	2023-01-20	Smartphon
5	108	1011	Electro Central	Los Angeles	11	99.99	2023-01-25	Wireless Mo
6	110	1013	Tech Haven	New York	5	59.99	2023-01-27	Portable Ch
7	107	1010	Tech Haven	New York	3	249.99	2023-01-24	Tablet
8	103	1004	Electro Central	Los Angeles	5	499.99	2023-01-18	High-End L
9	105	1008	Electro Central	Los Angeles	8	349.99	2023-01-22	Smartwatch

Resumamos lo que hicimos:

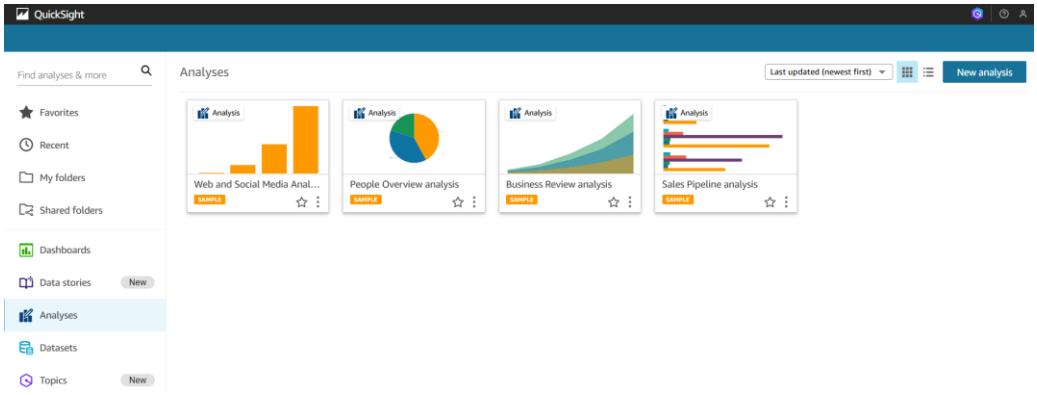
1- Cargamos la información en un bucket.
2- Con el crawler identificamos esta información y la cargamos en el AWS Glue datacatalog
3- El ETL extrajo la información
4- El ETL transformó la información, cambió los campos, unió las tablas
5- Cargamos la información en el bucket destino, le conectamos Athena para validar que todo el job de transformación de datos hubiera terminado de manera satifsfactoria.

Laboratorio de Amazon QuickSight

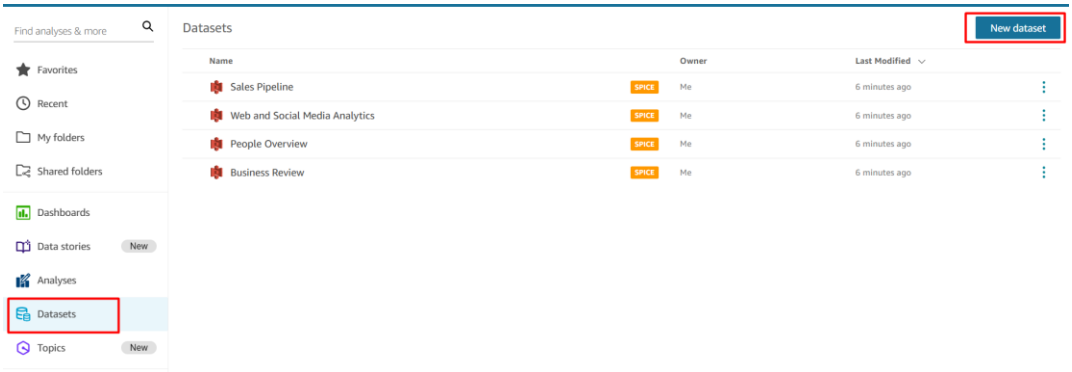
Arquitectura de referencia:

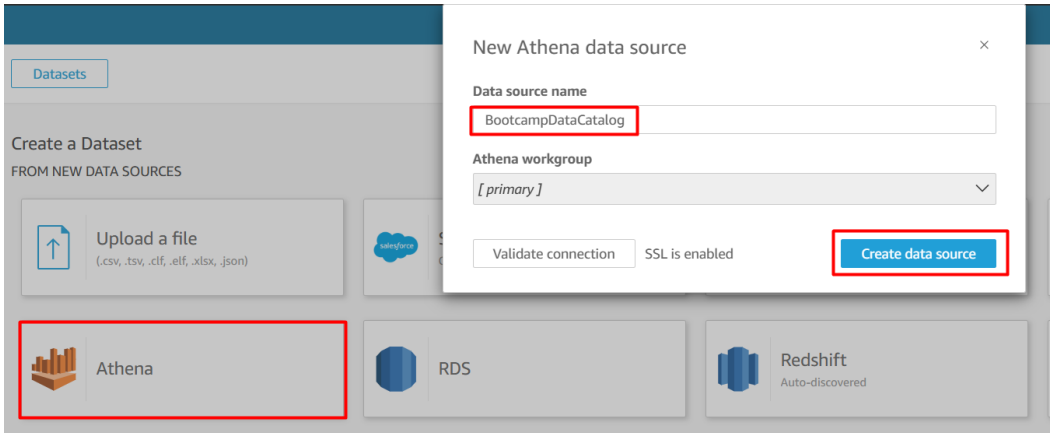


1. Ir a la consola de Amazon QuickSight, como es la primera vez debemos configurar la cuenta, una vez configurada podemos ingresar al panel principal donde ya podemos empezar a trabajar con nuestros datos:

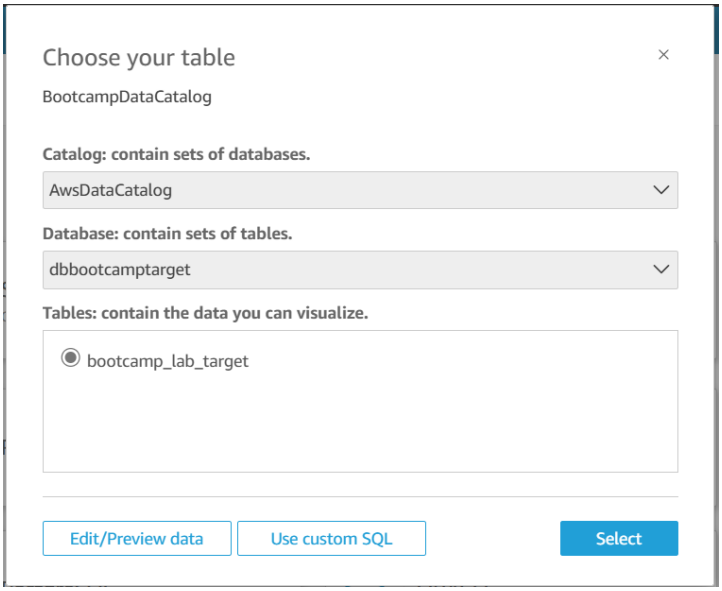


2. Crear dataset: Clic en New dataset, seleccionamos Athena, asignamos un nombre y clic en data source:





Seleccionamos la base de datos, una tabla y damos clic en Select:

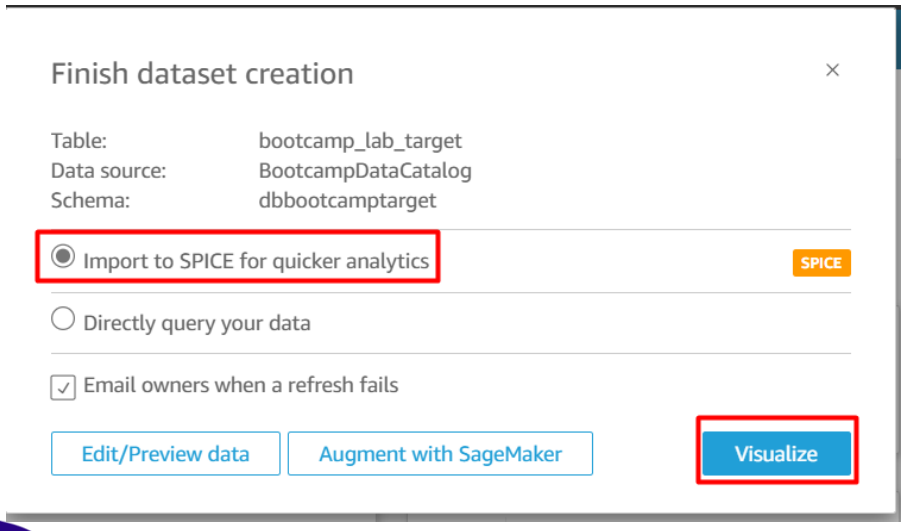


Tenemos dos opciones para importar la data:

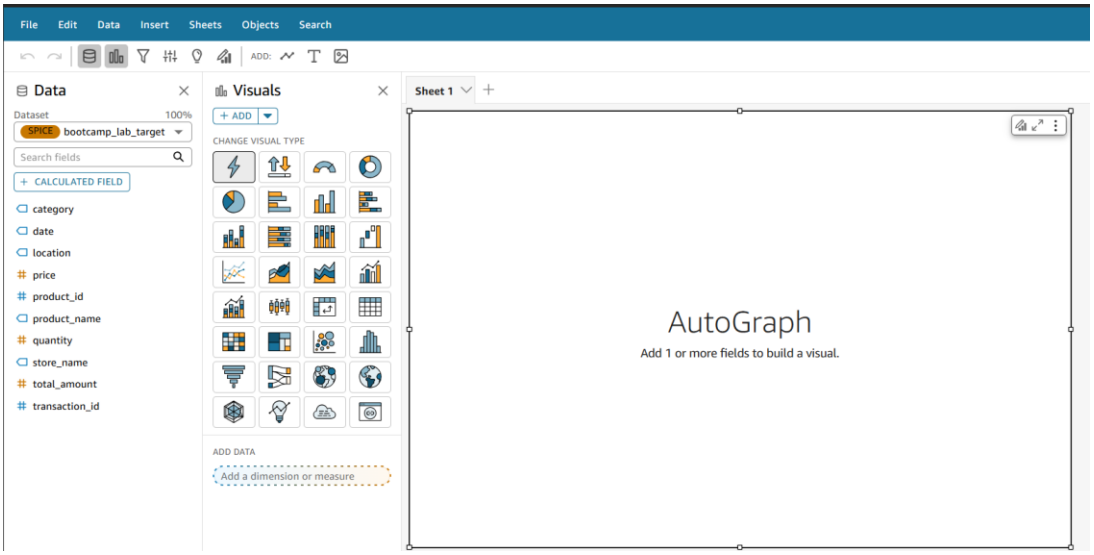
- Spice es un motor de cálculo que está en memoria, es decir, los datos van estar almacenados en la memoria para poder acceder a ellos de forma más rápida.

- Ir directamente a la fuente, la metadata me está diciendo dónde está la fuente, podemos ir a preguntarle a la fuente cada vez que cambiemos alguna visualización o queramos hacer un nuevo cálculo.

La forma más rápida es generar una copia con spice.



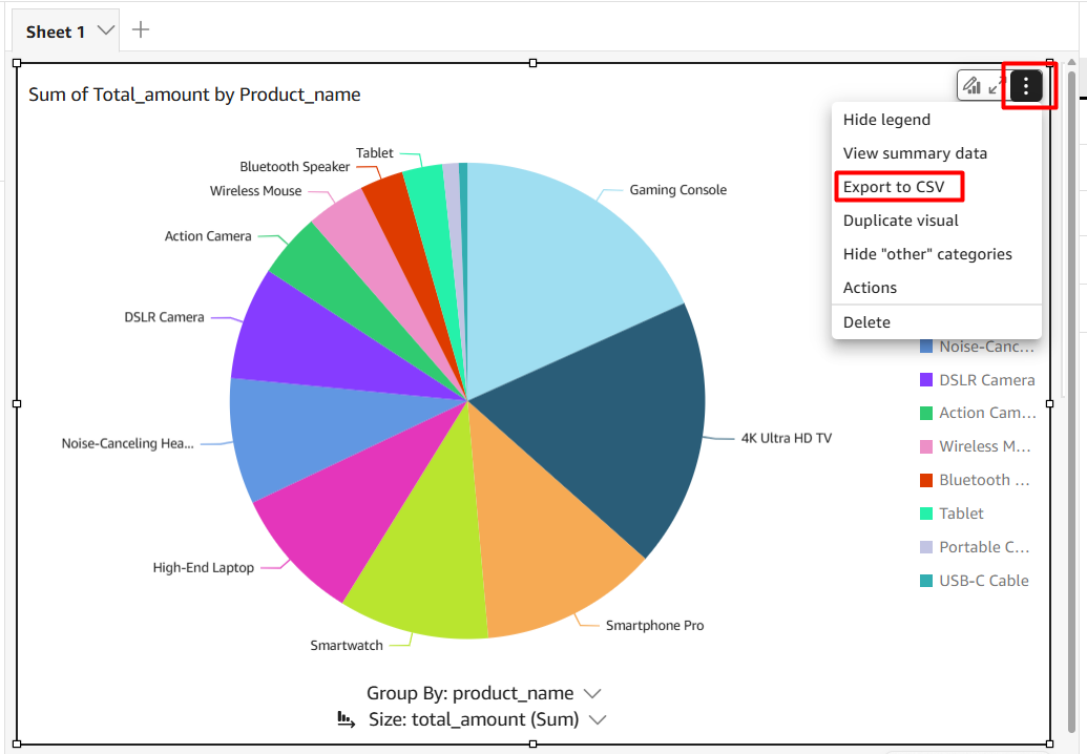
Damos clic en Visualize y nos traslada al panel principal de QuickSight:



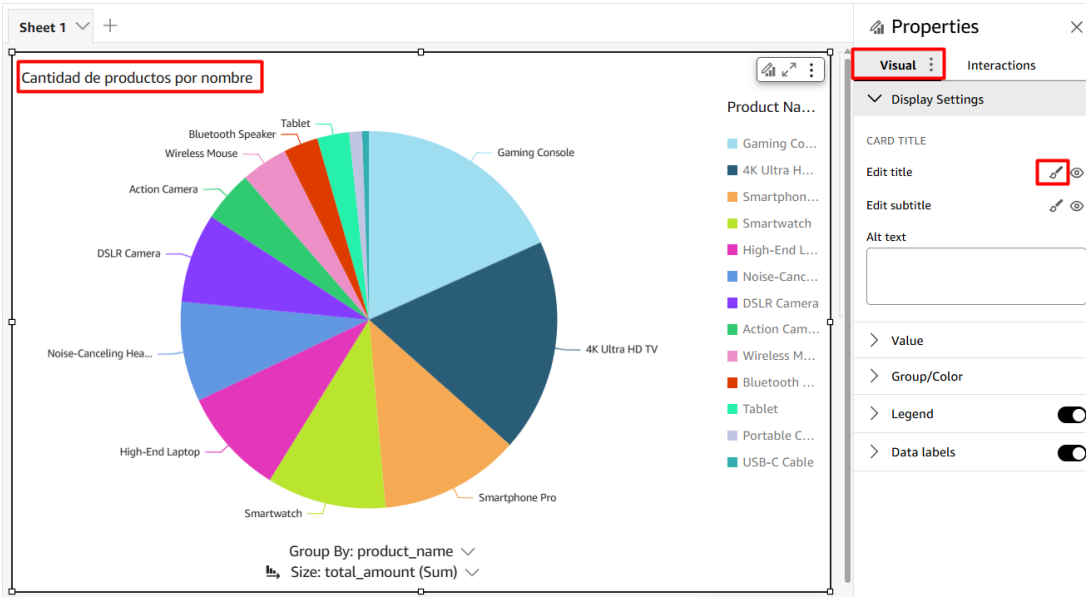
AuthoGraph significa que Amazon QuickSight va a seleccionar el mejor gráfico, dependiendo de los campos que le indiquemos, seleccionamos los campos a graficar:

Sin embargo, podemos cambiarlo y elegir otro si así lo deseamos.

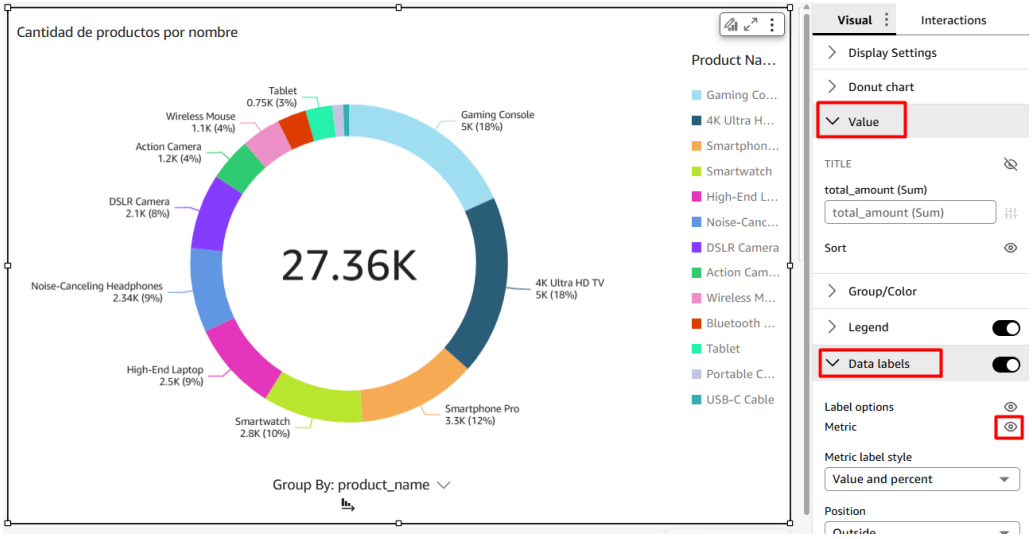
Podemos exportar el gráfico, duplicarlo y eliminarlo:



Podemos cambiarle el nombre, el tamaño, los colores:



Podemos ver los data labels o valores de los datos:



Ahora ya puedes explorar el servicio y personalizar los gráficos de acuerdo a las necesidades de tu negocio.

¡Felicitaciones! Obtuviste tu primera experiencia práctica con servicios de analítica, ahora recuerda hacer la limpieza de recursos para evitar costos innecesarios.